

1) Titolo e riferimenti dello studente

Applicazione di analisi e ottimizzazione dei dati di vendita musicali

Garruso Gianluca s311880

2) Copia integrale della proposta di progetto

Studente proponente

s311880 Gianluca Garruso

Titolo della proposta

Applicazione di analisi e ottimizzazione dei dati di vendita musicali

Descrizione del problema proposto

Il progetto ha l'obiettivo di sviluppare un'applicazione software in grado di analizzare dati di vendita musicali a partire da un database relazionale (Chinook), mediante la costruzione di modelli a grafo. In particolare si vogliono studiare le relazioni tra diverse entità (brani, artisti e clienti) per individuare pattern significativi nei comportamenti di acquisto. Su tali modelli verranno applicati algoritmi di analisi e ottimizzazione, con l'obiettivo di supportare decisioni di tipo gestionale, come la raccomandazione di prodotti, la segmentazione della clientela e l'identificazione di strategie di marketing efficaci. L'applicazione consentirà inoltre all'utente di filtrare i dati e configurare i parametri di analisi, permettendo un'esplorazione flessibile e mirata del fenomeno studiato.

Descrizione della rilevanza gestionale del problema

Si vuole analizzare i dati di vendita attraverso modelli a grafo che consentano di supportare decisioni strategiche in ambito aziendale, in particolare nei processi di marketing e gestione del catalogo prodotti. La possibilità di individuare relazioni tra articoli acquistati congiuntamente permette di sviluppare strategie di cross-selling e di definire bundle commerciali, con l'obiettivo di aumentare il valor medio delle transazioni. Inoltre, l'identificazione di cluster di artisti e di segmenti di clientela consente di progettare campagne di marketing mirate e più efficaci. Gli algoritmi di ottimizzazione (ricorsivi) permettono di selezionare in modo efficiente prodotti o clienti target, massimizzando la copertura del mercato e migliorando l'utilizzo delle risorse aziendali.

Descrizione dei data-set per la valutazione

Il progetto utilizza il database relazionale Chinook, un dataset che simula il funzionamento di una piattaforma di vendita di contenuti musicali digitali. Il database è strutturato secondo un modello relazionale e contiene informazioni relative a clienti, fatture, brani musicali, album, artisti, playlist e generi, impiegati.

Le principali entità coinvolte nel progetto sono:

- Customer: rappresenta i clienti
- Invoice e InvoiceLine: descrivono rispettivamente le fatture e il dettaglio degli acquisti effettuati
- Track: rappresenta i singoli brani musicali
- Album e Artist: permettono di risalire agli autori dei brani
- Genre: classifica i brani per tipologia musicale

Le relazioni tra queste tabelle consentono di ricostruire in modo completo il comportamento di acquisto dei clienti. In particolare, attraverso gli opportuni join, è possibile collegare ogni cliente agli artisti e ai brani acquistati.

Il database include anche ulteriori tabelle, quali Playlist, PlaylistTrack, MediaType e Employee che permettono rispettivamente di rappresentare collezioni di brani, l'associazione tra playlist e tracce e il formato dei contenuti digitali. Tali entità non vengono utilizzate direttamente nel progetto, in quanto non risultano rilevanti rispetto agli obiettivi di analisi dei comportamenti di acquisto, ma servono a fornire una visione più completa della struttura del database.

Il dataset risulta quindi adatto allo sviluppo del progetto in quanto consente di costruire modelli a grafo basati su relazioni significative tra entità come individuare brani acquistati congiuntamente, artisti che condividono lo stesso pubblico e clienti con comportamenti di acquisto simili.

Dal punto di vista applicativo, il dataset permette di simulare scenari realistici di analisi delle vendite e supporto delle decisioni, come la raccomandazione di prodotti, la segmentazione della clientela e la pianificazione delle strategie di marketing. Inoltre, la presenza di attributi temporali (InvoiceDate) e di classificazione (Genre) consente di introdurre filtri sui dati, rendendo possibile un'analisi più mirata e riducendo la complessità computazionale degli algoritmi

Descrizione preliminare degli algoritmi coinvolti

Il progetto prevede la costruzione di diversi modelli a grafo a partire dai dati estratti dal database relazionale Chinook. I grafi verranno generati partendo dai dati estratti mediante opportune query dal database Chinook. I dati verranno poi elaborati all'interno dell'applicazione, secondo il pattern MVC + DAO. I principali problemi algoritmici riguarderanno la costruzione dei grafi, l'analisi della loro struttura e lo sviluppo di algoritmi di ottimizzazione basati su ricorsione/backtracking.

-Il primo grafo sarà il grafo di co-acquisto tra brani, in cui i nodi rappresentano le tracce musicali e gli archi rappresentano il fatto che due brani siano stati acquistati all'interno della stessa ricevuta. Il peso del grafo indicherà il numero di co-occorrenze tra i 2 brani. Su questo grafo si potranno visualizzare le componenti connesse ed individuare brani

fortemente associati tra loro. Dal punto di vista algoritmico si svilupperà un algoritmo di raccomandazione, che, dato un brano iniziale scelto dall'utente, selezionerà un insieme di K brani correlati massimizzando il peso complessivo delle associazioni evitando chiaramente duplicazioni.

-Il secondo grafo sarà il grafo degli artisti, in cui i nodi rappresentano gli artisti e gli archi collegano due artisti che condividono almeno un certo numero K di clienti (scelto dall'utente). Il peso dell'arco rappresenterà invece il numero di clienti che hanno acquistato brani di entrambi gli artisti. Su tale grafo verranno mostrati le relazioni tra artisti, i cluster e i nodi maggiormente centrali. La parte di ottimizzazione consisterà nella selezione di un insieme di K artisti da promuovere, con l'obiettivo di massimizzare la funzione di score:

$\text{score} = \text{clienti_coperti} - X * \text{sovrapposizione}$

$\text{clienti coperti} = \text{numero di clienti raggiunti dagli artisti correnti}$

$X = \text{parametro scelto dall'utente (pensavo un valore di circa 0.5)}$

$\text{Sovrapposizione} = \text{somma dei pesi degli artisti correnti}$

In questo modo si vuole quindi selezionare un insieme di K artisti in modo da cercare di coprire al meglio il mercato ma allo stesso tempo penalizzare la promozione di 2 artisti con clienti troppo simili.

-Il terzo grafo sarà il grafo dei clienti, in cui i nodi rappresentano i clienti e gli archi collegano 2 clienti che hanno comportamenti di acquisto simili, ad esempio hanno acquistato un numero minimo di brani in comune. Il peso dell'arco rappresenterà invece il grado di similarità tra clienti (somma del numero di brani acquistati in comune dai 2 clienti). Su questo grafo sarà possibile individuare cluster di clienti simili ed eventuali outlier.

La parte algoritmica potrà riguardare la selezione di K clienti target per una campagna promozionale, con l'obiettivo di massimizzare la copertura del mercato.

In questo contesto, ogni cliente selezionato consente di raggiungere non solo se stesso, ma anche clienti a lui simili, identificati tramite gli archi del grafo. L'obiettivo sarà selezionare un insieme di k clienti in modo da massimizzare la funzione:

$\text{score} = \text{copertura} - X * \text{ridondanza}$

$\text{copertura} = \text{numero di clienti raggiunti (selezionati + simili)}$

$\text{ridondanza} = \text{similarità tra i clienti selezionati (somma dei pesi degli archi tra di essi)}$

In questo modo si vuole selezionare clienti che rappresentano segmenti diversi della clientela, massimizzando l'efficacia della campagna promozionale.

Gli algoritmi di ottimizzazione verranno sviluppati mediante ricorsione e backtracking, coerentemente con gli argomenti sviluppati durante il corso. Poiché tali problemi possono avere complessità computazionale elevata, verranno introdotti filtri e parametri configurabili dall'utente, come intervallo temporale (per andare a vedere e valutare anche eventuali trend temporali), genere musicale, soglie minime e valori di K elementi nelle ottimizzazioni. Tutto ciò avrà lo scopo di ridurre la dimensione dei grafi e di rendere gli algoritmi utilizzabili in tempi ragionevoli. Tali parametri potranno essere affinati in fase di sviluppo sulla base dei test effettuati.

Descrizione preliminare delle funzionalità previste per l'applicazione software

L'applicazione sarà sviluppata con un'interfaccia grafica che consente all'utente di analizzare dati di vendita di prodotti musicali attraverso diversi modelli a grafo (vedi descrizione sopra). L'utente potrà selezionare il tipo di grafo da costruire (co-acquisto, similarità tra artisti e clienti) e impostare una serie di parametri per filtrare e personalizzare l'analisi.

In particolare, sarà possibile definire periodi temporali selezionando le date direttamente tra quelle presenti nel database, filtri per i generi musicali e le soglie minime. Una volta configurati i parametri l'utente potrà procedere alla creazione dei grafi e visualizzarne le caratteristiche principali, come numero di nodi e archi, componenti connesse e/o altre proprietà strutturali. Saranno ovviamente gestiti opportunamente parametri mancanti e eventuali eccezioni mediante opportuni alert.

L'applicazione consentirà inoltre di eseguire algoritmi di ottimizzazione specifici per ciascun grafo. I risultati verranno mostrati in formato testuale all'interno dell'interfaccia, evidenziando le soluzioni trovate e i relativi valori delle funzioni obiettivo.

L'interfaccia sarà progettata per essere semplice e intuitiva, permettendo all'utente di modificare facilmente i parametri e ripetere le analisi su sottoinsiemi diversi del dataset, al fine di esplorare il comportamento del sistema in scenari differenti.

3) Descrizione dettagliata del problema affrontato

Il progetto sviluppato affronta il problema dell'analisi dei dati di vendita musicali mediante tecniche di modellazione a grafo, con l'obiettivo di supportare attività di analisi e decisione in ambito gestionale e commerciale. Il contesto applicativo di riferimento è rappresentato da una piattaforma di distribuzione di contenuti musicali digitali, simulata attraverso il database relazionale Chinook, contenente informazioni relative a clienti, fatture, brani musicali, album, artisti e generi musicali.

In un contesto aziendale reale, la semplice consultazione dei dati memorizzati nel database non consente di individuare facilmente relazioni significative tra prodotti, artisti e clienti. Per questo motivo è stato adottato un approccio basato sulla teoria dei grafi, che permette di modellare le relazioni presenti nei dati e di evidenziare pattern utili per comprendere il comportamento degli utenti e supportare eventuali strategie di marketing.

Il problema è stato suddiviso in tre sotto-problemi distinti:

- analisi delle relazioni di co-acquisto tra brani musicali;
- analisi della similarità tra artisti sulla base della clientela condivisa;
- analisi della similarità tra clienti sulla base dei comportamenti di acquisto.

Per ciascun sotto-problema è stato costruito un grafo specifico e sono stati sviluppati algoritmi di analisi e ottimizzazione dedicati.

Prima ancora della costruzione dei grafi, l'applicazione presenta una sezione iniziale di insight che fornisce una panoramica immediata del dataset. In particolare vengono mostrati il numero totale di clienti, fatture e tracce presenti nel database, oltre alle classifiche dei brani più acquistati, degli artisti più ascoltati, dei generi musicali più popolari, degli artisti con il maggior numero di clienti distinti e dei clienti che hanno effettuato la spesa complessiva più elevata. Sebbene tali informazioni non fossero espressamente richieste nella specifica iniziale, esse risultano utili per contestualizzare l'analisi e permettono all'utente di comprendere rapidamente le dimensioni e le caratteristiche principali della base dati con cui sta interagendo.

Per quanto riguarda gli input del sistema, l'applicazione consente all'utente di selezionare intervalli temporali, generi musicali, soglie minime e parametri utilizzati dagli algoritmi di ottimizzazione. Tutti gli input vengono preventivamente validati al fine di evitare situazioni di errore o eccezioni durante l'esecuzione del programma. In particolare vengono verificati la presenza dei valori richiesti e la correttezza degli intervalli temporali selezionati.

Gli output dell'applicazione comprendono sia informazioni descrittive sia risultati derivanti dagli algoritmi implementati. Per ogni grafo vengono mostrati il numero di nodi e di archi, oltre ai risultati delle analisi strutturali e delle procedure di ottimizzazione.

Nel primo grafo, dedicato ai co-acquisti tra brani musicali, i nodi rappresentano le tracce musicali mentre gli archi collegano due brani acquistati all'interno della stessa fattura. Il peso dell'arco rappresenta il numero di volte in cui i due brani sono stati acquistati congiuntamente. Oltre all'algoritmo di raccomandazione sviluppato mediante backtracking, è stata implementata un'analisi di centralità basata sulla betweenness centrality. Tale misura consente di individuare i brani che fungono da collegamento tra differenti aree del grafo e che quindi rivestono un ruolo particolarmente importante all'interno della rete di co-acquisto. Per ciascun brano vengono inoltre calcolati il grado e il grado pesato, fornendo una descrizione più completa della sua rilevanza.

Nel secondo grafo, dedicato agli artisti, i nodi rappresentano gli artisti presenti nel catalogo musicale e gli archi collegano artisti che condividono una parte della clientela. Anche in questo caso è stata effettuata un'analisi di centralità volta a identificare gli artisti maggiormente centrali all'interno della rete. Tali artisti possono essere interpretati come punti di aggregazione della domanda, in quanto risultano collegati a numerosi altri artisti tramite clienti comuni. Dal punto di vista gestionale questa informazione può risultare utile per attività promozionali, campagne di marketing e valorizzazione del catalogo.

Nel terzo grafo, dedicato ai clienti, i nodi rappresentano gli utenti della piattaforma mentre gli archi collegano clienti caratterizzati da comportamenti di acquisto simili. Oltre all'algoritmo di ottimizzazione per la selezione di clienti target, è stata sviluppata una procedura di analisi che distingue tra clienti rappresentativi e possibili outlier. I clienti rappresentativi sono quelli maggiormente connessi all'interno della rete e risultano quindi indicativi di specifici segmenti di mercato. Gli outlier, al contrario, presentano poche connessioni e rappresentano comportamenti di acquisto particolari o meno frequenti.

Questa distinzione consente di ottenere una prima forma di segmentazione automatica della clientela.

Dal punto di vista delle potenzialità, il sistema consente di supportare diverse attività decisionali. L'analisi dei co-acquisti permette di individuare relazioni significative tra prodotti e può essere utilizzata per strategie di cross-selling o per la costruzione di sistemi di raccomandazione. L'analisi degli artisti consente di identificare cluster di mercato e opportunità promozionali. L'analisi dei clienti permette invece di individuare segmenti omogenei e di selezionare target ottimali per campagne di marketing.

La principale criticità emersa durante lo sviluppo riguarda la complessità computazionale degli algoritmi di ottimizzazione basati su ricorsione e backtracking. In particolare, il problema risulta evidente nel secondo grafo, dove l'algoritmo deve valutare tutte le possibili combinazioni di artisti candidati. Considerando che il grafo può raggiungere circa cento nodi, il numero di combinazioni cresce rapidamente all'aumentare del parametro K. Durante la fase di test è stato osservato che valori di K superiori a 3 determinano tempi di elaborazione non compatibili con un utilizzo interattivo dell'applicazione. Per tale motivo si è scelto di limitare il numero massimo di artisti selezionabili a tre, garantendo comunque risultati significativi e tempi di risposta adeguati.

Un'ulteriore criticità è stata rappresentata dalla corretta trasformazione dei dati relazionali in strutture a grafo. Tale attività ha richiesto la progettazione di query SQL articolate e la definizione di opportuni criteri per la costruzione dei nodi, degli archi e dei relativi pesi.

Nel complesso il sistema sviluppato soddisfa gli obiettivi definiti nella proposta iniziale. Tutte le funzionalità previste sono state implementate e testate con successo. L'applicazione risulta stabile, gestisce correttamente gli input dell'utente e fornisce strumenti di analisi e ottimizzazione coerenti con il contesto gestionale considerato.

4)Descrizione del data-set utilizzato per l'analisi

Per lo sviluppo del progetto è stato utilizzato il database relazionale Chinook, un dataset ampiamente impiegato a scopo didattico per simulare il funzionamento di una piattaforma di vendita e distribuzione di contenuti musicali digitali.

Il database rappresenta un contesto realistico di e-commerce musicale e contiene informazioni riguardanti clienti, fatture, brani musicali, album, artisti, generi e altre entità correlate. La struttura del dataset risulta particolarmente adatta all'analisi dei comportamenti di acquisto poiché consente di ricostruire in modo completo le relazioni tra utenti e prodotti acquistati.

Dal punto di vista strutturale, il database è organizzato secondo un modello relazionale composto da diverse tabelle collegate mediante chiavi primarie e chiavi esterne. Le principali tabelle utilizzate nel progetto sono:

- Customer: contiene le informazioni anagrafiche dei clienti;
- Invoice: rappresenta le fatture emesse per ciascun acquisto;
- InvoiceLine: contiene il dettaglio dei prodotti acquistati all'interno di ogni fattura;
- Track: rappresenta i singoli brani musicali presenti nel catalogo;
- Album: permette di associare ogni brano al relativo album;
- Artist: contiene gli artisti musicali;
- Genre: classifica i brani secondo il genere musicale di appartenenza.

Le relazioni tra tali tabelle consentono di ricostruire l'intero processo di acquisto. In particolare, attraverso le tabelle Invoice e InvoiceLine è possibile associare ciascun cliente ai brani acquistati. Mediante le tabelle Album e Artist è inoltre possibile risalire agli artisti corrispondenti ai brani acquistati. La presenza della tabella Genre permette infine di effettuare analisi limitate a specifiche categorie musicali.

L'intero progetto si basa proprio sull'elaborazione di tali relazioni. I tre grafi sviluppati nell'applicazione vengono infatti costruiti a partire da opportune interrogazioni SQL che collegano tra loro le informazioni provenienti dalle diverse tabelle del database.

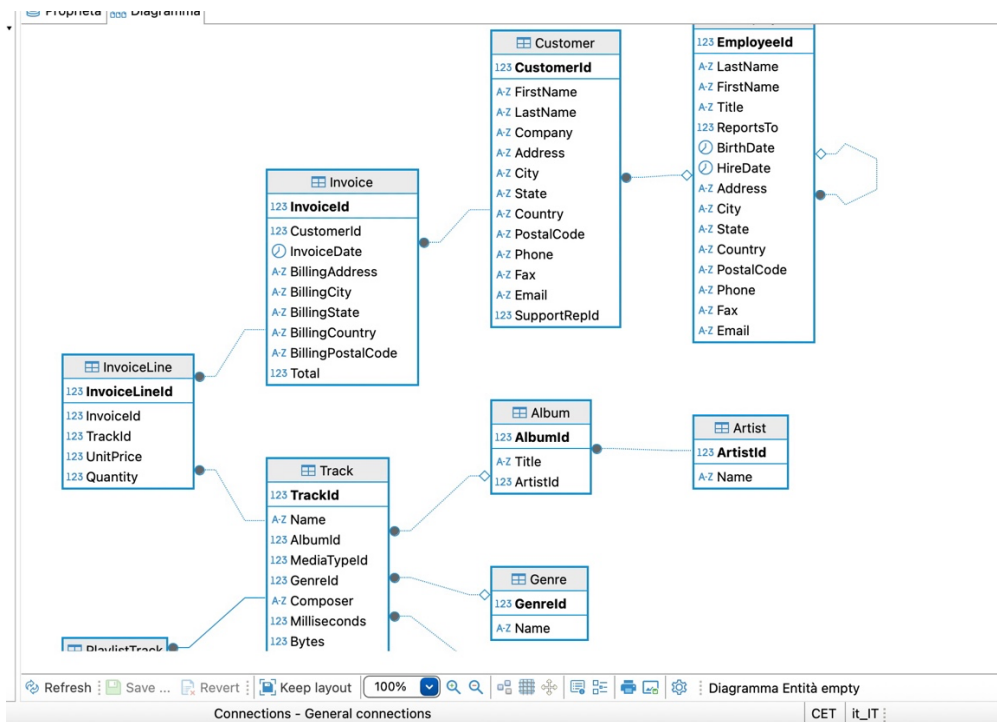
Il primo grafo utilizza principalmente le tabelle Invoice, InvoiceLine, Track e Genre per individuare i brani acquistati congiuntamente all'interno della stessa fattura. Il secondo grafo sfrutta invece le relazioni tra Customer, Invoice, InvoiceLine, Track, Album e Artist per identificare artisti che condividono parte della clientela. Il terzo grafo utilizza i dati di acquisto dei clienti per individuare comportamenti simili e costruire una rete di relazioni tra utenti.

Oltre alle tabelle utilizzate direttamente nell'applicazione, il database Chinook contiene ulteriori entità quali Playlist, PlaylistTrack, MediaType ed Employee. Tali tabelle non sono state considerate nello sviluppo del progetto poiché non risultavano rilevanti rispetto agli obiettivi prefissati, che riguardano principalmente l'analisi delle vendite e dei comportamenti di acquisto.

Il dataset si è dimostrato particolarmente adatto allo sviluppo del progetto per diversi motivi. Innanzitutto presenta una struttura sufficientemente complessa da consentire la costruzione di modelli a grafo significativi. Inoltre, la presenza di attributi temporali, come la data della fattura, e di attributi di classificazione, come il genere musicale, permette di introdurre filtri e di effettuare analisi mirate su specifici sottoinsiemi dei dati.

Dal punto di vista quantitativo, il database contiene migliaia di relazioni tra clienti, fatture e brani musicali, garantendo una base dati sufficientemente ampia per testare sia gli algoritmi di analisi sia le procedure di ottimizzazione implementate. Ciò ha consentito di

valutare il comportamento dell'applicazione in scenari realistici e di verificare la correttezza dei risultati ottenuti.



Screenshot del database con le tabelle usate nell'applicazione.

5) Descrizione ad alto livello delle strutture dati e degli algoritmi utilizzati

Dopo aver definito il dataset di riferimento e le relazioni presenti nel database Chinook, è stato necessario progettare un insieme di strutture dati e algoritmi in grado di trasformare le informazioni relazionali in modelli utilizzabili per l'analisi. L'applicazione sviluppata utilizza una combinazione di strutture dati tradizionali, quali liste e dizionari, e strutture più avanzate basate sulla teoria dei grafi.

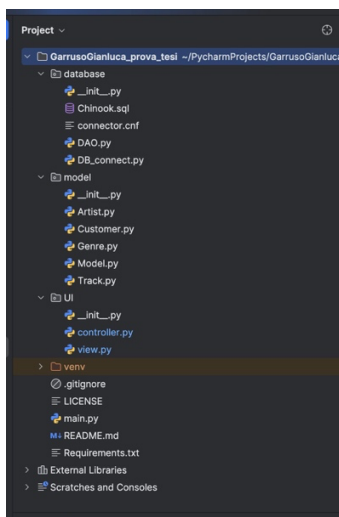
Le liste vengono impiegate per memorizzare nodi, archi e soluzioni parziali durante l'esecuzione degli algoritmi, mentre i dizionari consentono un accesso rapido agli oggetti tramite il relativo identificativo proveniente dal database. L'elemento centrale dell'applicazione è tuttavia rappresentato dai grafi implementati mediante la libreria NetworkX, scelti per la loro capacità di modellare relazioni complesse tra entità diverse.

Oltre alla costruzione dei grafi, il progetto prevede l'utilizzo di algoritmi ricorsivi di ottimizzazione basati sulla tecnica del backtracking, finalizzati all'individuazione delle migliori soluzioni secondo specifiche funzioni obiettivo. Sono inoltre presenti procedure di analisi strutturale che sfruttano misure di centralità e proprietà topologiche dei grafi per estrarre informazioni significative dai dati.

Nei paragrafi seguenti vengono descritte le principali strutture dati utilizzate e gli algoritmi implementati all'interno dell'applicazione.

-Struttura del progetto:

Il progetto è stato organizzato seguendo una struttura modulare, separando le diverse responsabilità applicative e rispettando il pattern MVC+DAO. La cartella database contiene i file relativi alla connessione al database e al DAO, responsabile dell'esecuzione delle query SQL. La cartella model contiene le classi dataclass utilizzate per rappresentare le principali entità del database e il file Model.py, in cui sono implementate la logica applicativa, la costruzione dei grafi e gli algoritmi. La cartella UI contiene invece la View e il Controller, responsabili rispettivamente dell'interfaccia grafica e della gestione degli eventi utente.



-Metodo __init__

Nel costruttore della classe Model vengono inizializzate le principali strutture dati utilizzate dall'applicazione. Per ciascuno dei tre grafi sono presenti una lista di nodi, una lista di archi e un dizionario di supporto che associa l'identificativo numerico del database all'oggetto Python corrispondente.

I tre grafi sono rappresentati mediante oggetti `nx.Graph()` della libreria NetworkX. Questa scelta consente di gestire in modo semplice nodi, archi pesati, vicini di un nodo e misure di centralità.

I dizionari `idMapNodes` sono particolarmente importanti perché permettono di convertire rapidamente gli identificativi estratti dalle query SQL negli oggetti usati come nodi del grafo.

```

7
8 class Model: 2 usages  ⤴ Gianluca Garruso
9     def __init__(self):  ⤴ Gianluca Garruso
10         self._edges_3 = []
11         self._idMapNodes_3 = {}
12         self._nodes_3 = []
13         self._artista_cliente = {}
14         self._idMapNodes_2 = {}
15         self._edges_2 = []
16         self._nodes_2 = []
17         self._edges_1 = []
18         self._idMapNodes_1 = {}
19         self._nodes_1 = []
20         self._graph1 = nx.Graph()
21         self._graph2 = nx.Graph()
22         self._graph3 = nx.Graph()
23         #self._idMap_copertura_artisti_clienti = DAO.get_copertura_artisti_clienti()
24
25

```

-Metodo buildGraph1

Il metodo buildGraph1 si occupa della costruzione del primo grafo, relativo ai co-acquisti tra brani musicali. Il grafo viene prima svuotato, in modo da eliminare eventuali risultati di elaborazioni precedenti, e successivamente vengono estratti dal DAO i nodi e gli archi coerenti con il genere musicale e l'intervallo temporale scelti dall'utente.

I nodi rappresentano i brani musicali, mentre gli archi collegano due brani acquistati nella stessa fattura. Il peso dell'arco indica quante volte i due brani sono stati acquistati insieme. Dopo l'estrazione dei dati, i nodi vengono aggiunti al grafo tramite add_nodes_from, mentre gli archi vengono inseriti singolarmente con add_edge, specificando il relativo peso.

```

def buildGraph1(self, genre, date1, date2): 1 usage (1 dynamic)  ⤴ Gianluca Garruso
    self._graph1.clear()
    self._nodes_1 = DAO.get_nodes_1(genre, date1, date2)
    self._idMapNodes_1 = {}
    for node in self._nodes_1:
        self._idMapNodes_1[node.TrackId] = node
    self._edges_1 = DAO.get_edges_1(genre, date1, date2, self._idMapNodes_1)
    self._graph1.add_nodes_from(self._nodes_1)
    for item in self._edges_1:
        self._graph1.add_edge(item[0], item[1], weight=item[2])

```

-Metodo buildGraph2

Il metodo buildGraph2 costruisce il grafo degli artisti. Anche in questo caso il grafo viene prima svuotato e poi ricostruito sulla base dei parametri selezionati dall'utente.

I nodi rappresentano gli artisti, mentre gli archi collegano due artisti che condividono almeno un certo numero di clienti, definito dalla soglia scelta. Il peso dell'arco rappresenta il numero di clienti comuni tra i due artisti. Questa struttura permette di modellare la similarità tra artisti sulla base del comportamento reale degli utenti.

```
def buildGraph2(self, soglia, date1, date2): 1 usage (1 dynamic)  ⤴ Gianluca Garruso
    self._graph2.clear()
    self._nodes_2 = DA0.get_nodes_2(date1, date2)
    self._idMapNodes_2 = {}
    for node in self._nodes_2:
        self._idMapNodes_2[node.ArtistId] = node
    self._graph2.add_nodes_from(self._nodes_2)
    self._edges_2 = DA0.get_edges_2(soglia, date1, date2, self._idMapNodes_2)
    for edge in self._edges_2:
        self._graph2.add_edge(edge[0], edge[1], weight=edge[2])
```

-Metodo buildGraph3

Il metodo buildGraph3 costruisce il grafo dei clienti. I nodi rappresentano i clienti presenti nel database, mentre gli archi collegano clienti con comportamenti di acquisto simili. La soglia indicata dall'utente stabilisce il numero minimo di brani in comune necessario per creare una relazione tra due clienti.

Anche in questo caso viene utilizzato un dizionario di supporto per associare l'identificativo del cliente all'oggetto corrispondente. Il grafo risultante permette di analizzare la struttura della clientela e individuare clienti rappresentativi o comportamenti anomali.

```
def buildGraph3(self, soglia): 1 usage (1 dynamic)  ⤴ Gianluca Garruso
    self._graph3.clear()
    self._nodes_3 = DA0.get_nodes_3()
    self._idMapNodes_3 = {}
    for node in self._nodes_3:
        self._idMapNodes_3[node.CustomerId] = node
    self._graph3.add_nodes_from(self._nodes_3)
    self._edges_3 = DA0.get_edges_3(soglia, self._idMapNodes_3)
    for edge in self._edges_3:
        self._graph3.add_edge(edge[0], edge[1], weight=edge[2])
```

-Metodo analyze_customer_segments

Per ogni grafo ho anche implementato dei metodi come il seguente che vanno ad analizzare il grafo andando a mostrare quelli che sono i nodi più rilevanti soprattutto

usando il criterio di centralità e grado del nodo in questione. Lo screen sotto mostra il metodo usato nel grafo 3, che va ad analizzare i customers e ritorna una lista dei clienti più rappresentativi e una lista degli outlier.

```
return list(self._graph1.nodes)

def analyze_customer_segments(self, outlier_threshold=1):
    representative_customers = []
    outlier_customers = []

    for customer in self._graph3.nodes:
        degree = self._graph3.degree(customer)
        weighted_degree = self._graph3.degree(customer, weight="weight")

        data = {
            "customer": customer,
            "degree": degree,
            "weighted_degree": weighted_degree
        }
        if degree <= outlier_threshold:
            outlier_customers.append(data)
        else:
            representative_customers.append(data)

    representative_customers.sort(key=lambda x: x["weighted_degree"], reverse=True)
    outlier_customers.sort(key=lambda x: x["degree"])
    return representative_customers[:3], outlier_customers
```

-Metodo get_optPath_coAcquisto e ricorsione_coAcquisto

Questi metodi implementano l'algoritmo di ottimizzazione relativo al primo grafo. L'utente seleziona un brano di partenza e un valore K, cioè il numero di brani da includere nel percorso.

L'algoritmo utilizza una procedura ricorsiva che esplora i vicini del nodo corrente e costruisce progressivamente una soluzione parziale. Quando la soluzione raggiunge lunghezza K, viene calcolato lo score tramite un metodo ausiliario. Se lo score ottenuto è migliore del migliore trovato fino a quel momento, la soluzione viene salvata.

Il metodo utilizza quindi una logica di backtracking: aggiunge un nodo alla soluzione parziale, prosegue ricorsivamente e poi lo rimuove per esplorare alternative diverse. I metodi ausiliari, come calcolaScore1, completano la funzionalità valutando il peso complessivo del percorso individuato.

```

def get_optPath_coAcquisto(self, brano_start_id, k_brani): 1 usage (1 dynamic)  ⚡ Gianluca Garruso
    brano_start = self._idMapNodes_1[brano_start_id]
    correnti = [brano_start]
    self.bestScore1 = 0
    self.bestSolution_coAcquisto = []
    self.ricorsione_coAcquisto(correnti, k_brani)
    return self.bestSolution_coAcquisto, self.bestScore1

def ricorsione_coAcquisto(self, correnti, k): 2 usages  ⚡ Gianluca Garruso
    if len(correnti) == k:
        score = self.calcolaScore1(correnti)
        if score > self.bestScore1:
            self.bestSolution_coAcquisto = copy.deepcopy(correnti)
            self.bestScore1 = score
        return
    ultimo = correnti[-1]
    vicini = list(self._graph1.neighbors(ultimo))
    for node in vicini:
        if node not in correnti:
            correnti.append(node)
            self.ricorsione_coAcquisto(correnti, k)
            correnti.pop()

```

-Metodo get_optPath_Artisti e ricorsione_artisti

Questi metodi implementano l'algoritmo di ottimizzazione relativo al secondo grafo. L'obiettivo è selezionare un insieme di K artisti da promuovere, massimizzando una funzione di score che tiene conto sia della copertura della clientela sia della sovrapposizione tra gli artisti scelti.

L'algoritmo esplora le possibili combinazioni di artisti tramite ricorsione e backtracking. Per ogni combinazione completa viene calcolato uno score attraverso metodi ausiliari, tra cui calcolaScore2, calcolaCopertura2 e calcolaSovrapposizione2.

Questa parte rappresenta una delle componenti algoritmiche più rilevanti del progetto, ma anche una delle più critiche dal punto di vista computazionale, poiché il numero di combinazioni cresce rapidamente all'aumentare del valore di K.

```

def get_optPath_Artisti(self,k_artisti,alfa): 1 usage (1 dynamic)  ⬆ Gianluca Garruso
    if k_artisti > len(self._nodes_2):
        return [], 0
    self._best_artists = []
    self._best_score2 = float('-inf')
    res_query = DAO.get_artista_clienti()
    self.calcola_clienti_per_artista(res_query)

    self.ricorsione_artisti(k_artisti, parziale: [], alfa, start: 0)

    return self._best_artists, self._best_score2

def ricorsione_artisti(self,k_artisti,parziale,alfa,start): 2 usages  ⬆ Gianluca Garruso
    if len(parziale) == k_artisti:
        score = self.calcolaScore2(parziale,alfa)
        if score > self._best_score2:
            self._best_artists = copy.deepcopy(parziale)
            self._best_score2 = score
        return
    for i in range(start,len(self._nodes_2)):
        node = self._nodes_2[i]
        if node not in parziale:
            parziale.append(node)
            self.ricorsione_artisti(k_artisti, parziale, alfa,i+1)
            parziale.pop()

```

-Metodo getOpt_Path_Clienti e _ricorsione_clienti

Questi metodi implementano l'algoritmo di ottimizzazione relativo al terzo grafo. L'obiettivo è selezionare K clienti target per una possibile campagna promozionale.

La logica è simile a quella utilizzata per il grafo degli artisti: l'algoritmo costruisce ricorsivamente combinazioni di clienti e valuta ciascuna soluzione tramite una funzione di score. Tale funzione considera la copertura dei clienti raggiunti e la ridondanza tra i clienti selezionati. Anche in questo caso i metodi ausiliari completano la funzionalità principale, calcolando la copertura e la sovrapposizione tra i clienti scelti.

```

def getOpt_Path_Clienti(self,k_clienti,alfa): 1 usage (1 dynamic)  ⬆ Gianluca Garruso
    self._best_clients = []
    self._best_score_clients = float('-inf')

    self._ricorsione_clienti(k_clienti, parziale: [],alfa, start: 0)

    return self._best_clients, self._best_score_clients

def _ricorsione_clienti(self,k_clienti,parziale,alfa,start): 2 usages  ⬆ Gianluca Garruso
    if len(parziale) == k_clienti:
        score = self.calcolaScore_clienti(parziale,alfa)
        if score > self._best_score_clients:
            self._best_clients = copy.deepcopy(parziale)
            self._best_score_clients = score
        return
    nodes = list(self._graph3.nodes)
    for i in range(start,len(nodes)):
        node = nodes[i]
        if node not in parziale:
            parziale.append(node)
            self._ricorsione_clienti(k_clienti,parziale,alfa,i+1)
            parziale.pop()

```

6) Diagramma delle classi delle parti principali dell'applicazione

Il diagramma delle classi riportato in questa sezione rappresenta in forma semplificata le componenti principali dell'applicazione, con particolare attenzione alle classi coinvolte nella parte algoritmica.

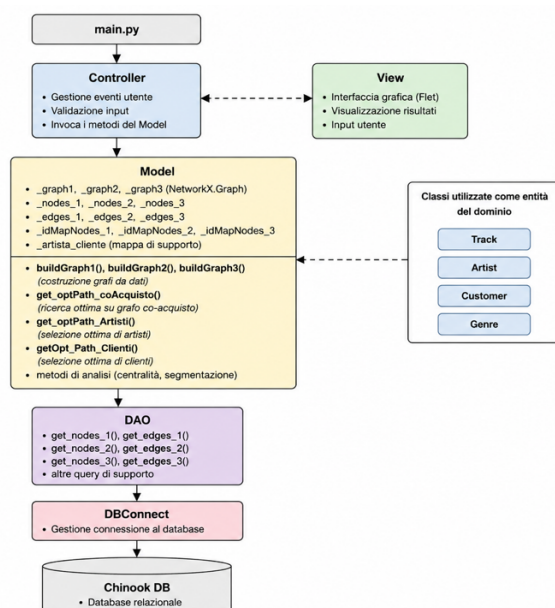
Rispetto al capitolo precedente, in cui sono state descritte le strutture dati e i principali algoritmi implementati, questa sezione ha l'obiettivo di mostrare le relazioni tra le classi e il flusso logico dell'applicazione.

L'applicazione è organizzata secondo una struttura riconducibile al pattern MVC, con separazione tra interfaccia utente, logica applicativa e accesso ai dati.

La classe View gestisce la costruzione dell'interfaccia grafica e la visualizzazione dei risultati. La classe Controller riceve gli input dell'utente, effettua i controlli preliminari e richiama i metodi opportuni del Model. La classe Model rappresenta il nucleo algoritmico dell'applicazione: al suo interno vengono costruiti i tre grafi, memorizzate le principali strutture dati e implementati gli algoritmi di analisi e ottimizzazione.

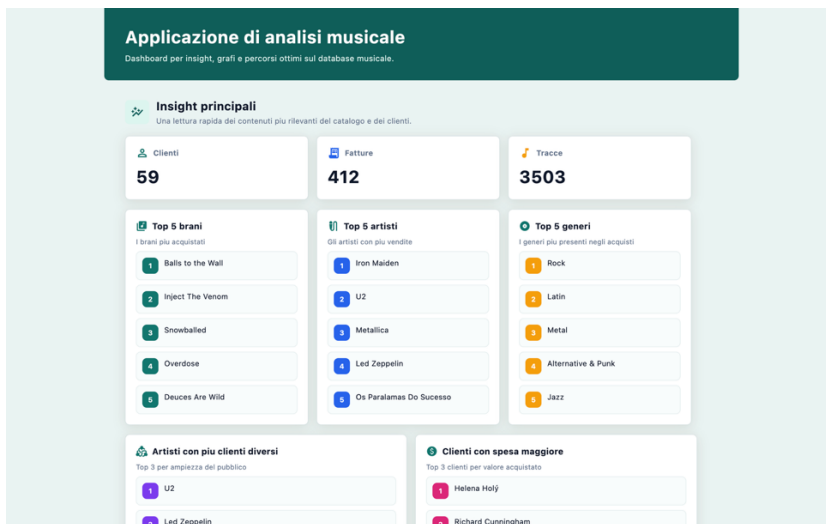
La classe DAO si occupa dell'estrazione dei dati dal database Chinook tramite query SQL dedicate. Essa restituisce al Model nodi e archi già trasformati in oggetti Python. La classe DBConnect gestisce invece la connessione al database.

Le classi Track, Artist, Customer e Genre rappresentano le principali entità del dominio applicativo. In particolare, Track, Artist e Customer vengono utilizzate come nodi rispettivamente nei tre grafi implementati.

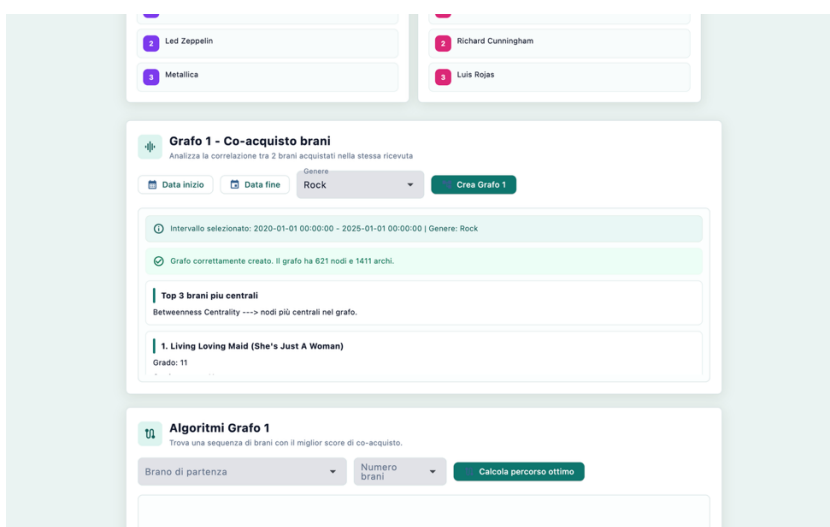


7) Alcune videate dell'applicazione realizzata e link al video dimostrativo del software

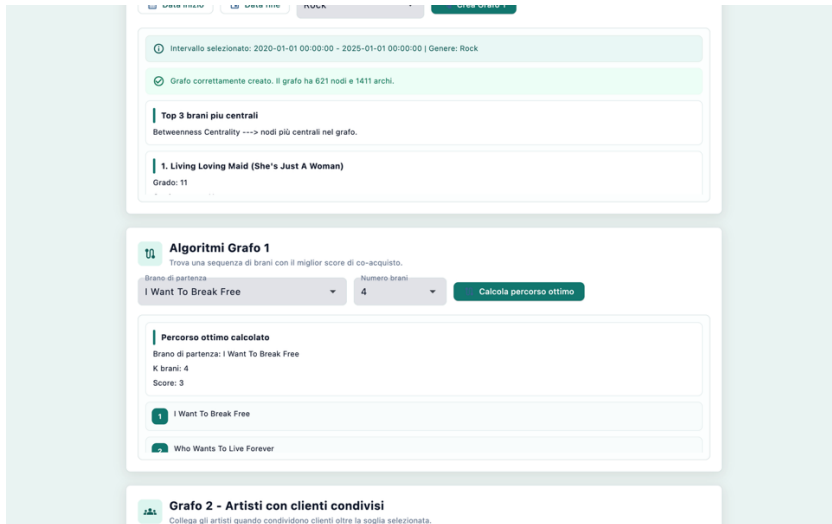
-Schermata iniziale dell'applicazione che mostra i dati maggiormente rilevanti della base dati come si può vedere dall'immagine sotto allegata. Vengono mostrati rispettivamente: numero di clienti, numero di fatture, numero di tracce, top 5 brani, top 5 artisti, top 5 generi, top 3 artisti con più clienti diversi e top 3 clienti che hanno speso di più.



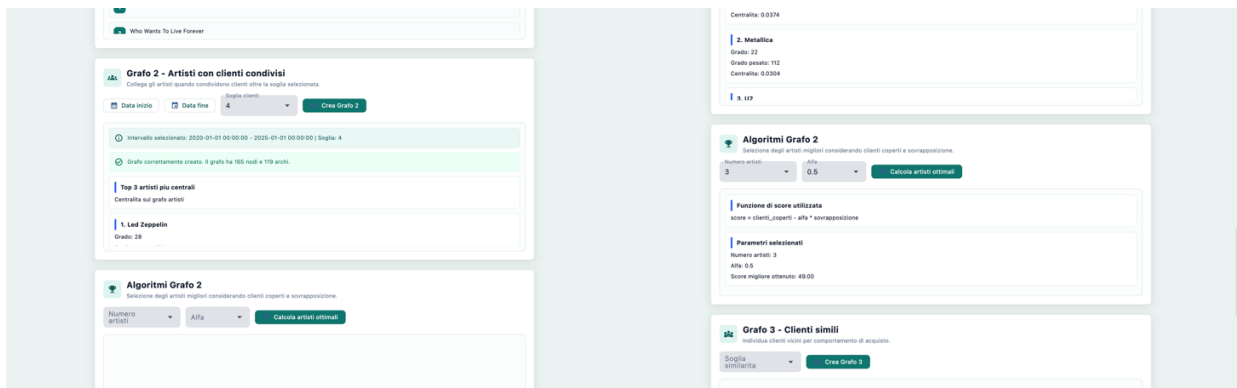
-Schermata che mostra il funzionamento del grafo 1, vengono infatti visualizzati a schermo i parametri utilizzati dall'utente e la corretta creazione del grafo con le rispettive informazioni principali che lo riguardano. Per quanto riguarda i parametri ho scelto di mettere due datepicker per selezionare l'intervallo temporale e un dropdown per scegliere il genere. In caso di parametri mancanti la creazione del grafo non avviene e viene mostrato a schermo un alert per segnalare il problema.



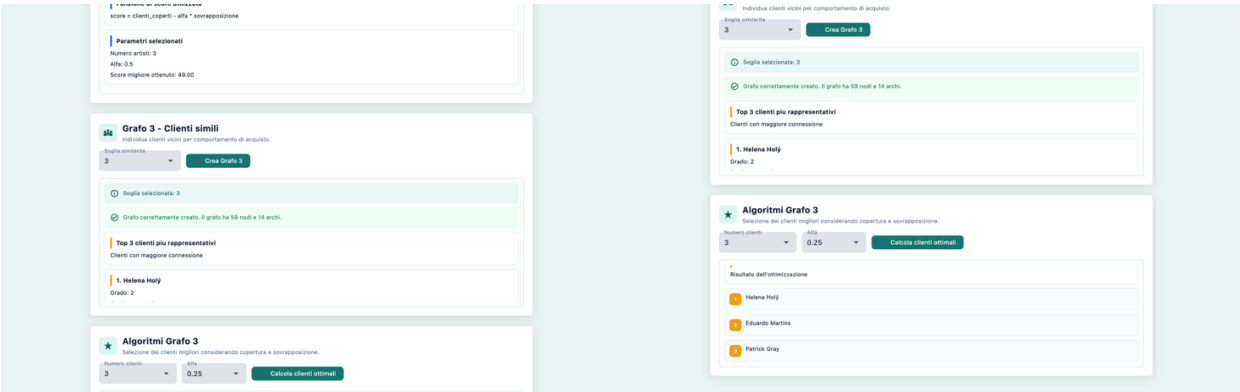
-Schermata che mostra la parte algoritmica del grafo 1; come si può vedere il dropdown del brano di partenza viene riempito solo dopo l'avvenuta creazione del grafo. Dopo che l'utente sceglie il numero di k_brani da consigliare, l'applicazione mostra i risultati nella schermata con score e lista dei k_brani consigliati.



-Schermate che mostrano il funzionamento del grafo 2 con la sua rispettiva parte algoritmica, molto simile al grafo 1.



-Schermate che mostrano il funzionamento del grafo 3, con la parte del grafo e quella algoritmica.



Link del video YouTube: <https://youtu.be/FSL-5HIdP-s>

8) Tabelle con risultati sperimentali ottenuti

-Risultati grafo 1

Data inizio	Data fine	Genere	Num nodi	Num archi
2020/01/01	2025/01/01	Rock	621	1411
2020/01/01	2025/01/01	Jazz	54	51
2022/01/01	2025/01/01	Pop	25	23
2023/01/01	2024/01/01	Rock	163	410

-Risultati grafo 2

Data inizio	Data fine	Soglia	Num nodi	Num archi
2020/01/01	2025/01/01	2	165	1117
2022/01/01	2025/01/01	2	165	638
2023/01/01	2025/01/01	3	153	16
2024/01/01	2026/01/01	2	143	173

-Risultati grafo 3

Soglia	Num nodi	Num archi
1	59	199
2	59	43

3	59	14
---	----	----

-Risultati centralità canzoni (grafo 1)

Date 2020/01/01-2025/01/01 - genere: Rock

Canzone	Degree	Weighted Degree	Betweenness
Living Loving Maid	11	11.0	0.0002
Inject The Venom	10	10.0	0.0002
Two Hearts Beat As One	9	9.0	0.0001

-Risultati centralità artisti (grafo 2)

Date 2020/01/01-2025/01/01 – soglia clienti: 2

Artista	Degree	Weighted Degree	Betweenness
Metallica	77	249	0.0748
Led Zeppelin	77	273	0.0646
U2	70	245	0.0480

-Risultati centralità clienti

Soglia: 2

Nome cliente	Degree	Weighted Degree
Eduardo Martins	3	8
Mark Philips	3	8
Marta Silk	3	8

Parte algoritmica

Algoritmo	Parametri testati	Esito
Co-acquisto	K= 2,3,4,5,6	corretto
Ottimizzazione artisti	K = 2,3	corretto
Ottimizzazione artisti	K >= 4	tempi elevati
Ottimizzazione clienti	K = 2,3,4,5	corretto

9)Valutazioni sui risultati ottenuti e conclusioni.

L'obiettivo del progetto era sviluppare un'applicazione in grado di analizzare i dati di vendita contenuti nel database Chinook attraverso tecniche di modellazione a grafo e algoritmi di ottimizzazione. I risultati ottenuti dimostrano che l'applicazione è in grado di costruire correttamente i tre grafi previsti, eseguire analisi strutturali e individuare soluzioni ottimali in funzione dei parametri scelti dall'utente.

Uno dei principali punti di forza del progetto è rappresentato dalla capacità di trasformare un database relazionale in una rappresentazione a grafo che consente di evidenziare relazioni non immediatamente osservabili mediante semplici interrogazioni SQL. Attraverso i grafi è infatti possibile individuare connessioni tra brani musicali, artisti e clienti, ottenendo informazioni utili per comprendere il comportamento degli utenti e supportare eventuali decisioni di natura commerciale.

Un ulteriore punto di forza è rappresentato dalla flessibilità dell'applicazione. L'utente può modificare soglie, intervalli temporali e parametri degli algoritmi ottenendo analisi differenti a partire dalla stessa base dati. La presenza di controlli sugli input e di una interfaccia grafica intuitiva rende inoltre il software facilmente utilizzabile anche da utenti privi di competenze tecniche specifiche.

Dal punto di vista algoritmico, il progetto integra sia tecniche di analisi dei grafi sia algoritmi di ottimizzazione basati su ricorsione e backtracking. Le funzionalità di centralità, segmentazione e ricerca delle migliori soluzioni consentono di ottenere informazioni aggiuntive rispetto a quelle richieste inizialmente nella specifica del progetto, aumentando il valore informativo dell'applicazione.

La principale limitazione emersa durante lo sviluppo riguarda la complessità computazionale dell'algoritmo di ottimizzazione implementato per il secondo grafo. L'esplorazione delle possibili combinazioni di artisti comporta infatti una crescita molto rapida del numero di soluzioni da valutare. Durante le attività di test è stato osservato che valori di K superiori a 3 determinano tempi di elaborazione non compatibili con un utilizzo interattivo. Per questo motivo si è scelto di limitare tale parametro all'interno dell'interfaccia grafica.

Nonostante questa limitazione, l'applicazione risulta pienamente funzionante e produce risultati coerenti con gli obiettivi prefissati. Tutte le funzionalità previste sono state implementate e sottoposte a verifica, correggendo progressivamente le criticità emerse durante la fase di sviluppo e test.

Con riferimento al problema descritto nel contesto aziendale, il contributo principale del progetto consiste nella capacità di estrarre conoscenza dai dati di vendita attraverso strumenti di analisi avanzata. Le informazioni ottenute potrebbero essere utilizzate per

attività di raccomandazione di prodotti, individuazione di artisti strategici, segmentazione della clientela e supporto alle decisioni di marketing.

In conclusione, il progetto ha consentito di applicare in modo concreto le principali tematiche affrontate durante il corso di Tecniche di Programmazione, tra cui progettazione software, utilizzo di basi di dati relazionali, strutture dati, grafi, algoritmi ricorsivi e sviluppo di interfacce grafiche. I risultati ottenuti possono essere considerati soddisfacenti sia dal punto di vista funzionale sia dal punto di vista didattico, avendo permesso la realizzazione di un'applicazione completa e coerente con gli obiettivi inizialmente definiti.

Licenza

Quest'opera è distribuita con licenza Creative Commons Attribution – NonCommercial – ShareAlike 4.0 International (CC BY-NC-SA 4.0).

